

18.1.1: void

From time to time we have seen the keyword `void` lurking in various programs and code samples. `void` is sort of a "placeholder" type, used in circumstances where you need a type name but there's not really any one right type to use. Formally, we can say that `void` is a type with no values.

There are three main uses of the `void` type:

1. As the return type of a function which does not return a value. A function declared as

```
void f()
```

is declared as "void" or "returning void" which actually means that it returns no value. The compiler will not complain if you "fall off the end" of a `void` function without executing a `return` statement; the compiler *will* complain if you execute a `return` statement that specifies a value to be returned. (As far as the low-level syntax of the `return` statement is concerned, the expression is optional; but the expression is required in functions that return values and disallowed in `void` functions.)

2. As the argument list in the prototype of a function that accepts no parameters. In a function definition such as

```
int f()
{
    ...
}
```

the empty parentheses indicate that the function accepts no parameters. But (for historical reasons) in an external function prototype declaration such as

```
extern int f();
```

the empty parentheses indicate that we don't know how many (or what type of) parameters the function accepts. In either case, we can make the fact that the function accepts zero parameters explicit by using the keyword `void` in the parameter list:

```
extern int f(void);

int f(void)
{
    ...
}
```

For obvious reasons, if `void` appears in a parameter list, it must be the first and only parameter, and it must not declare an argument name. (That is, prototypes like `int f(int, void)` and `int f(void x)` are meaningless and illegal.)

3. As a pointer type, to indicate a "generic pointer" which might in fact point to any type. We need "generic pointers" when we're using functions like `malloc`. `malloc` returns a pointer to *n* bytes of memory, which we may use as any type of pointer we wish. Normally, it is an error to use a value of one pointer type where another pointer type is required. For example, the fragments

```
int i;
double *dp = &i;          /* WRONG */
```

and

```
int *ip = dp;                /* WRONG */
```

would both generate errors, because you can't assign back and forth between `int` pointers and `double` pointers. However, the type `void *` ("pointer to `void`") is special: it *is* legal to assign a value of type pointer-to-`void` to a variable of some other pointer type, and vice versa. (In case the pointer types have different sizes or representations, the compiler will automatically perform conversions. We'll say more about type conversions, including pointer type conversions, in a later section.) So the lines

```
#include <stdlib.h>

char *cp = malloc(10);
int *ip = malloc(sizeof(int));
double *dp = malloc(sizeof(double));
```

are all legal, since `<stdlib.h>` declares `malloc` as returning `void *`, indicating that an assignment of `malloc`'s return value to any pointer type is permissible.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1996-1999 // [mail feedback](#)